Lab 4: Vibe Coding

Frogger - completed submission package

STUDENT	Pratham P Mudakavi
SRN	PES2UG24CS369
ASSIGNED REPOSITORY	github.com/SETAPESU26/01_frogger
STATUS	All four implementation tasks and all three handout deliverables completed

SUBMISSION READY

The package contains the updated source code, an exact 10-second before video, an exact 10-second after video, and this complete AI interaction record.

Prepared 30 September 2026

1. Deliverables

Deliverable	Location in Lab-4	Verification
Before video	Videos/01_before_changes.mp4	10.0 s, 30 fps, 640 x 560
After video	Videos/02_after_changes.mp4	10.0 s, 30 fps, 640 x 560
Updated code	Updated_Code/frogger/	Runs with Python 3.10+ and Pygame
Chat history	Chat_History/Chat_History_and_Lab_Report.pdf	This document

2. Requirement completion

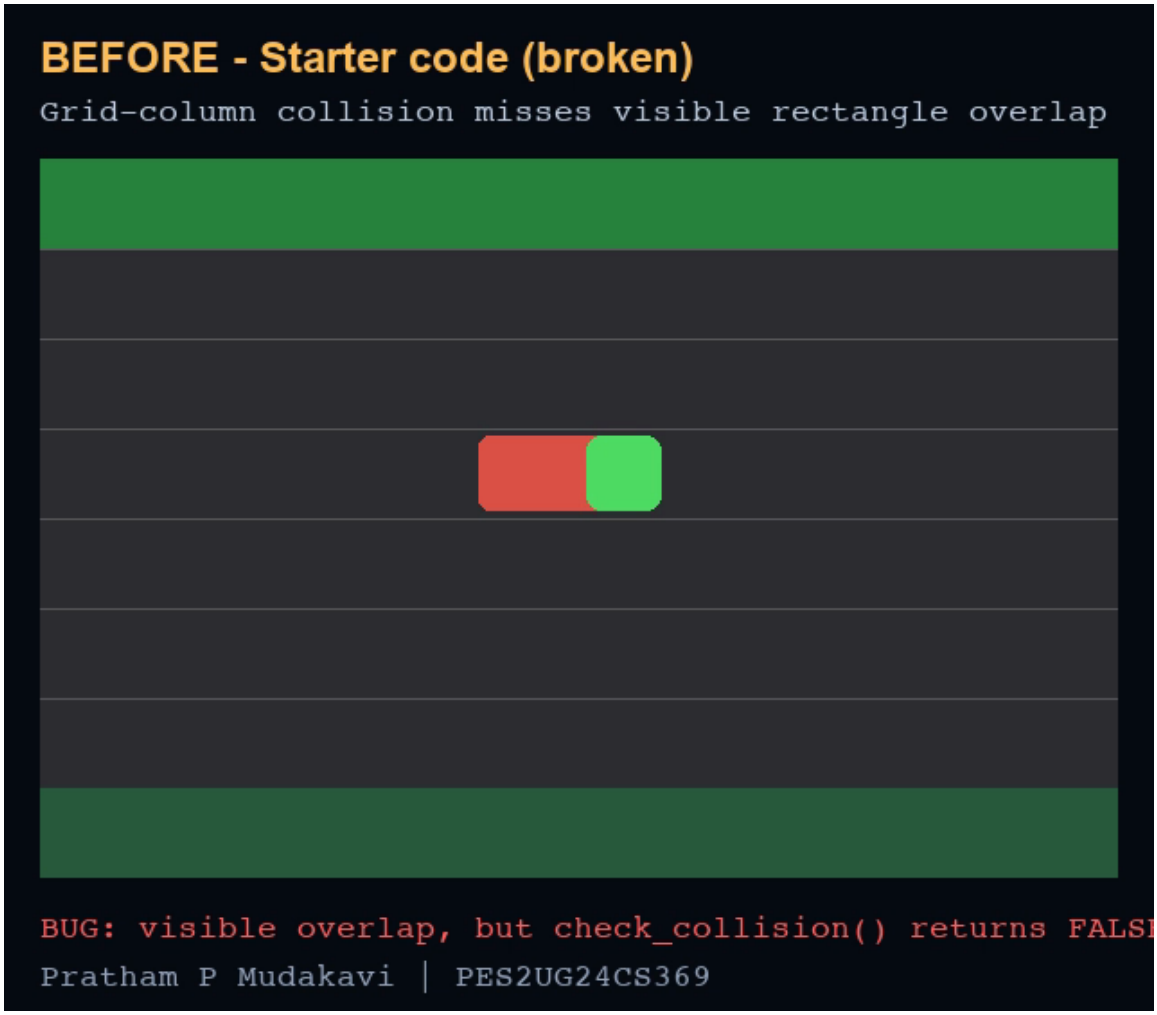
Task	Implemented behavior	Result
1. Collision	Uses colliderect() on the frog and vehicle pixel rectangles, including cross-cell overlaps.	PASS
2. Lives	Starts at three lives. Collision decrements one life, shows a 750 ms hit state, then respawns. Third loss ends the game.	PASS
3. Goal and score	Goal increments score once and enters a persistent You Won! state.	PASS
4. Timer	Each attempt starts at 30 seconds. Timeout costs a life and uses the same visible failure/respawn flow.	PASS
Restart	R resets entities, lives, score, timer, and terminal state.	PASS

Automated verification

Seven unit tests passed: boundary-overlap collision, safe non-overlap, collision life loss and delayed respawn, goal score/win, timeout penalty, three-failure game over, and full restart.

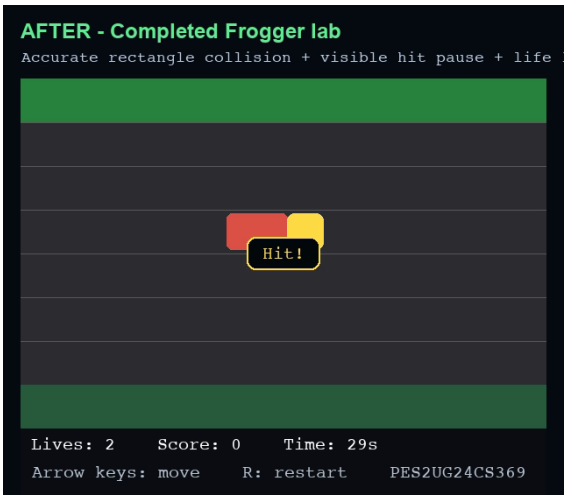
3. Video evidence

The before capture reproduces the original grid-column bug: the wide vehicle and frog overlap on screen while the old check still returns false.

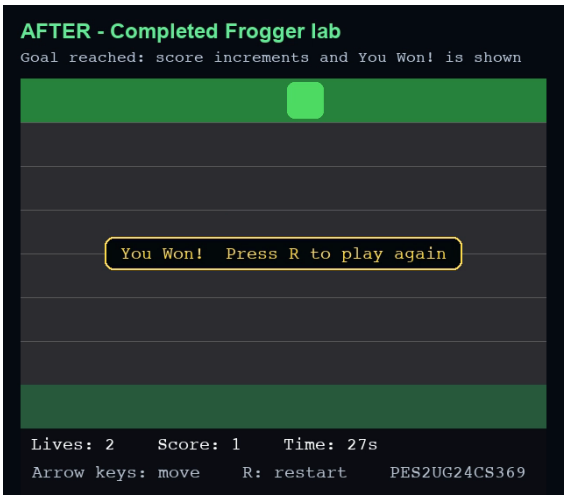


Before: visible collision overlap is missed by the starter implementation.

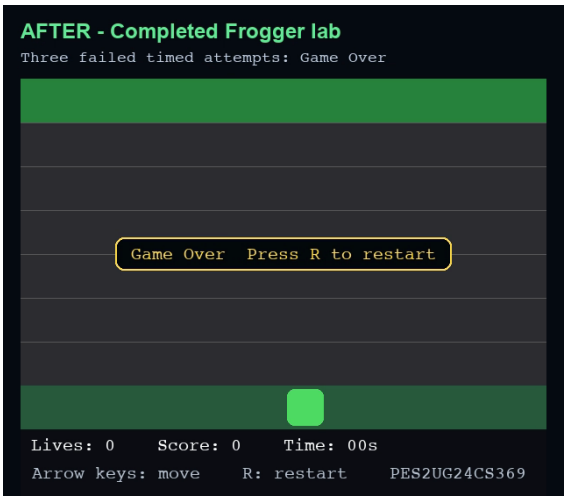
3. Video evidence - completed game



Hit is visible; lives drop from 3 to 2.



Goal increments score and shows You Won!



Three timeouts end the game with lives at 0.

The video also shows the 30-second timer expiring, each timeout costing a life, respawn between attempts, and Game Over after all three lives are used.

All state transitions are driven by the implemented GameEngine, not by a static mock-up.

4. Complete visible chat history

USER - INITIAL PROMPT

https://github.com/SETAPESU26/01_frogger

this is the lab i should do so complete the final requirement also give me all the deliverables done and comepleted

SRN- PES2UG24CS369
Name- Pratham P Mudakavi

ASSISTANT - RESPONSE 1

I will inspect the lab instructions and the current workspace, then implement the final requirement and assemble every required deliverable using your name and SRN. I will verify the result against the repository's stated submission checklist.

ASSISTANT - RESPONSE 2

The workspace currently contains only the lab handout, so I am using the PDF workflow to extract and visually verify its requirements before creating the solution. I will also inspect the linked GitHub repository for any rubric or starter files not duplicated in the handout.

ASSISTANT - RESPONSE 3

The repository confirms four concrete tasks: rectangle-based collisions, three lives with a visible hit pause, score/win tracking, and a 30-second per-attempt timer where timeout costs a life. The submission itself needs before/after 10-second videos plus the complete AI-chat record; I am building a deterministic, testable version and will generate those artifacts locally rather than leaving placeholders.

ASSISTANT - RESPONSE 4

The core implementation is complete and all seven automated checks pass, including the original cross-grid-boundary collision case, visible hit/respawn timing, score/win, timeout penalties, game over, and full restart. I am now producing the two required 10-second MP4 demonstrations from deterministic game states so each rubric item is visibly provable.

ASSISTANT - RESPONSE 5

Both MP4s are now verified at exactly 10.0 seconds, 30 fps, and 640 x 560. The before video visibly demonstrates the missed overlap and absent systems; the after video shows life loss with a hit pause, a scored win, timeout penalties, respawn, and game over. I am packaging the code and the PDF/chat-link deliverable now.

5. Implementation record

Problem analysis

The starter compared int(vehicle.x // CELL_SIZE) with frog.col. A 70-pixel truck can begin in column 5 and overlap a frog drawn in column 6, so the old equality is false during a real visual collision. The fix derives rectangles from the same get_rect() methods used by the renderer and calls pygame.Rect.colliderect().

State design

The completed engine uses four explicit states: playing, hit, won, and game_over. Inputs move the frog only while playing; R remains available in every state. The hit state keeps the frog at the impact position for 750 ms so the collision is visible. Respawn resets the per-attempt timer, while the score and remaining lives persist.

Timer semantics

Each attempt starts with 30 seconds. If time reaches zero, the engine invokes the same life-loss path used by collisions. The goal condition is evaluated before timeout, so reaching the goal at the final instant correctly counts as a win.

How to reproduce

Action	Command
Install dependency	python -m pip install -r requirements.txt
Run the game	python main.py
Run tests	python -m unittest discover -s tests -v

FINAL RESULT

The implementation, evidence videos, test suite, and AI interaction record are complete and organized under the Lab-4 directory.